

ENGENHARIA DE SOFTWARE — ANÁLISE E PROJETO DE SOFTWARE

Análise e Projeto de Software

*Arquitetura em Camadas, Diagramas UML, Padrões GoF, Princípios SOLID/GRASP,
Métricas de Qualidade e Log de Refatoração*

Sistema: Gestão de Vendas de Veículos

Tecnologia: Spring Boot 4.0.6 | Java 21 | REST API

Equipe: NextCar

Documento: Arquitetura do projeto

Resumo da Aplicação

O **NextCar** é uma aplicação backend REST para gestão de vendas de veículos. O sistema foi organizado com arquitetura em camadas, separação de responsabilidades, uso de DTOs, padrões GoF, princípios SOLID/GRASP, testes automatizados e refatorações voltadas à qualidade de software.

Este documento segue a seguinte sequência: **Introdução e Contexto**, **Visão de Projeto UML**, **Memorial de Padrões e Princípios**, **Arquitetura e Qualidade** e **Log de Refatoração**.

Equipe de Desenvolvimento

Projeto acadêmico desenvolvido para a disciplina de Análise e Projeto de Software

Função	Integrante
Desenvolvedor	Emerson Willian do Nascimento Dias
Desenvolvedor	Lucas Hudson Caxilé
Desenvolvedora	Marcela Cristine Cantalice Santos
Desenvolvedor	Marcos Dalyson dos Santos Borba
Desenvolvedor	Mateus José Bispo Oliveira
Desenvolvedor	Rosalvo Alves de Oliveira Filho

Sumário

1	Introdução e Contexto	4
1.1	Visão Geral	4
1.2	Escopo	4
2	Visão de Projeto UML	4
2.1	Diagrama de Casos de Uso	4
2.2	Diagrama de Classes	6
2.3	Diagrama de Classes	8
2.4	Diagramas de Sequência — Usuário	8
2.4.1	Cadastro de Usuário — POST /usuarios	9
2.4.2	Atualização de Usuário — PUT /usuarios/id	10
2.4.3	Consultar Usuários — GET /usuarios	11
2.4.4	Remoção de Usuário — DELETE /usuarios/id	12
2.5	Diagramas de Sequência — Veículo	12
2.5.1	Cadastro de Veículo — POST /veiculos	13
2.5.2	Atualização de Veículo — PUT /veiculos/id	14
2.5.3	Consultar Veículos — GET /veiculos	15
2.5.4	Remoção de Veículo — DELETE /veiculos/id	16
2.6	Diagramas de Sequência — Venda	16
2.6.1	Processamento de Venda — POST /vendas	17
2.6.2	Atualização de Status de Venda — PUT /vendas/id	18
2.6.3	Consultar Vendas — GET /vendas	19
2.6.4	Remoção de Venda — DELETE /vendas/id	20
2.7	Diagramas de Colaboração — Veículo	21
2.8	Diagramas de Colaboração — Venda	23
2.9	Diagramas de Colaboração — Usuário	25
3	Memorial Descritivo de Padrões e Princípios	27
3.1	Tabela de Padrões GoF	27
3.2	Análise SOLID/GRASP	28
4	Arquitetura e Qualidade	28
4.1	Descrição Arquitetural	28
4.2	Relatório de Métricas	29
4.3	Plano de Testes	31
4.3.1	Scripts de Testes Unitários	31
4.3.2	Scripts de Testes de Integração (End-to-End)	48

5	Log de Refatoração	53
5.1	Antes vs. Depois	53
5.2	Exemplo 1: Modelo Exposto como DTO	54
5.3	Exemplo 2: Exceção Tipada	55
5.4	Exemplo 3: @Component em Interface	56
5.5	Exemplo 4: Controller Desacoplado com Facade	57
5.6	Exemplo 5: Magic String com if/else	59
5.7	Exemplo 6: @Valid Ausente no update	60
6	Conclusão	61

1 Introdução e Contexto

1.1 Visão Geral

O **NextCar** é um sistema backend desenvolvido com *Spring Boot 4.0.6* e *Java 21*, criado para gerenciar vendas de veículos de forma organizada, segura e extensível. A aplicação resolve o problema de registrar transações comerciais, controlar veículos disponíveis, associar usuários às vendas e calcular preços de acordo com estratégias comerciais.

1.2 Escopo

O foco da análise de projeto está nos módulos funcionais e arquiteturais mais importantes para a sustentação do sistema.

Tabela 1: Escopo Funcional do Projeto

Módulo	Descrição	Relevância
Vendas	Registra transações, calcula o preço final e atualiza o status do veículo.	Crítica
Usuários	Gerencia informações de usuários utilizadas nas vendas e operações do sistema.	Alta
Veículos	Controla cadastro, consulta, precificação e disponibilidade da frota.	Crítica
Precificação	Aplica estratégias de preço normal e desconto de 10% utilizando o padrão Strategy.	Alta
API REST	Disponibiliza endpoints para integração com aplicações clientes e usuários finais.	Alta
Persistência	Armazena entidades utilizando repositórios do Spring Data JPA.	Crítica

2 Visão de Projeto UML

Esta seção apresenta os campos UML: **casos de uso**, **classes**, **sequência** e **colaboração**.

2.1 Diagrama de Casos de Uso

Objetivo do Diagrama
Representar as funcionalidades principais do sistema sob a perspectiva dos atores que interagem com a aplicação.

Tabela 2: Casos de Uso — Sistema NextCar

Módulo	Ator	Caso de Uso	Descrição
Autenticação	Usuário	Fazer Login	Autentica o usuário no sistema.
Autenticação	Usuário	Gerenciar Credenciais	Permite alterar ou recuperar credenciais de acesso.
Gestão de Veículos	Usuário	Criar Veículo	Cadastra um novo veículo na plataforma.
Gestão de Veículos	Usuário	Editar Veículo	Atualiza informações de um veículo existente.
Gestão de Veículos	Usuário	Remover Veículo	Exclui um veículo do sistema.
Gestão de Veículos	Usuário	Cadastrar Veículo (<i>inclui</i>) Criar Veículo	Fluxo completo de cadastro que inclui a criação do veículo.
Precificação	Sistema	Aplicar Preço Normal	Mantém o preço base do veículo sem alterações.
Precificação	Sistema	Aplicar Desconto	Reduz o preço conforme a regra de desconto configurada.
Gestão de Vendas	Usuário	Registrar Venda	Realiza a venda, calcula o preço final e atualiza o status.
Gestão de Vendas	Usuário	Acessar Histórico de Vendas	Consulta transações registradas anteriormente.
Funções do Usuário	Usuário	Visualizar Catálogo	Lista os veículos disponíveis para venda.
Funções do Usuário	Usuário	Consultar Preço do Veículo	Exibe o preço calculado de um veículo específico.
Funções do Usuário	Usuário	Solicitar Venda	Inicia o processo de compra de um veículo.
Funções do Usuário	Usuário	Buscar Veículo por Id	Localiza um veículo a partir do seu identificador.
Funções do Usuário	Usuário	Ver Detalhes do Veículo	Exibe todas as informações de um veículo selecionado.



Figura 1: Diagrama de Casos de Uso — Sistema NextCar

2.2 Diagrama de Classes

Objetivo do Diagrama

Evidenciar as classes de negócio, DTOs, serviços, repositórios, interfaces e padrões GoF aplicados no projeto.

Tabela 3: Elementos do Diagrama de Classes

Classe/Interface	Tipo	Responsabilidade	Relações
Usuario	Entidade JPA	Representa um usuário do sistema.	Relaciona-se com Venda.
Veiculo	Entidade JPA	Representa um veículo vendido ou disponível.	Relaciona-se com Venda.
Venda	Entidade JPA	Representa uma transação comercial.	Agrega Usuario e Veiculo.
VendaController	Controller REST	Recebe requisições relacionadas às vendas.	Utiliza VendaFacade.
VendaFacade	Facade / Service	Orquestra o fluxo completo de venda.	Utiliza services e factory.
VendaService	Service	Persiste informações de vendas.	Utiliza VendaRepository.
UsuarioService	Service	Consulta e valida usuários.	Utiliza UsuarioRepository.
VeiculoService	Service	Consulta e atualiza veículos.	Utiliza VeiculoRepository.
PrecoStrategy	Interface	Define o contrato de cálculo de preço.	Implementada pelas estratégias concretas.
PrecoNormal	Strategy	Mantém o preço original do veículo.	Implementa PrecoStrategy.
PrecoDesconto	Strategy	Aplica desconto de 10%.	Implementa PrecoStrategy.
PrecoStrategyFact	Factory Method	Seleciona a estratégia de preço dinamicamente.	Utiliza um mapa de PrecoStrategy.

2.3 Diagrama de Classes

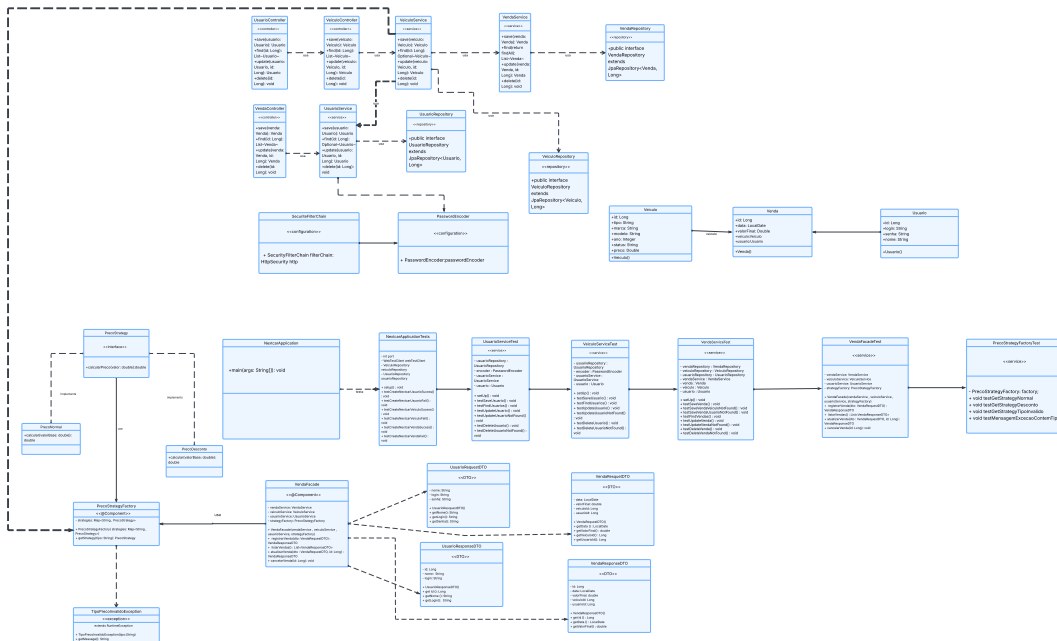


Figura 2: Diagrama de Classes — Sistema NextCar

2.4 Diagramas de Sequência — Usuário

Objetivo dos Diagramas

Detalhar a ordem temporal das mensagens trocadas entre os objetos do módulo de **Usuário** para cada operação CRUD da API REST.

2.4.1 Cadastro de Usuário — POST /usuarios

Tabela 4: Sequência: Cadastro de Usuário

	Origem	Destino	Mensagem
1	Cliente HTTP	UsuarioController	POST /usuarios
1.1a	UsuarioController: Cliente HTTP		400 BAD REQUEST (dados inválidos)
1.1b	UsuarioController: UsuarioRequestDTO		new UsuarioRequestDTO()
2	UsuarioController: UsuarioService		save(dto)
3	UsuarioService	UsuarioRepository	«create» Usuario / save(usuario)
4	UsuarioRepository: UsuarioService		Usuário persistido com senha criptografada
5	UsuarioService	UsuarioResponseDTO	new UsuarioResponseDTO()
6	UsuarioController: Cliente HTTP		UsuarioResponseDTO / 201 CREATED

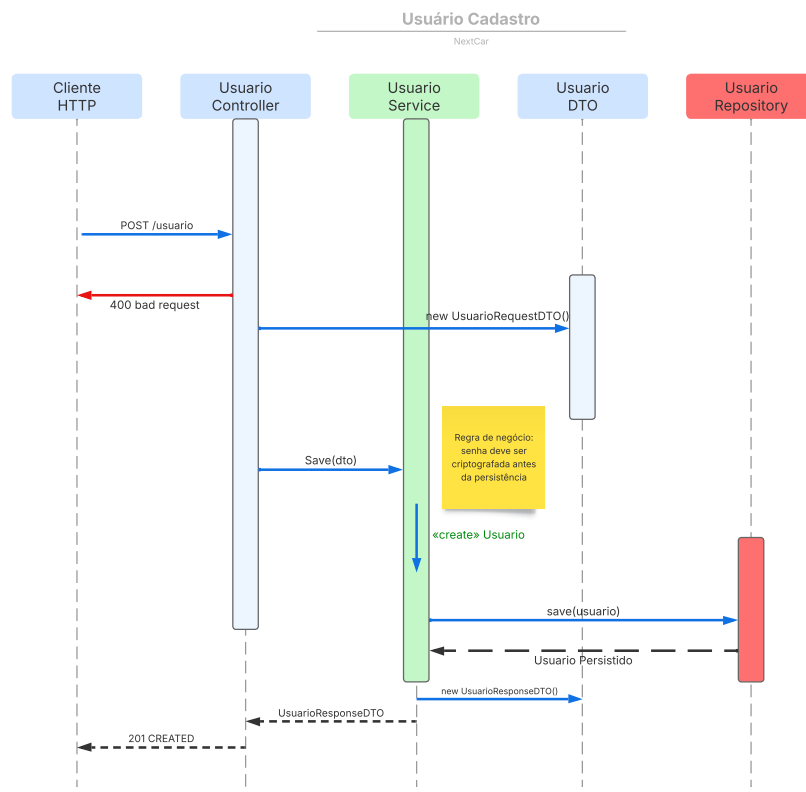


Figura 3: Diagrama de Sequência — Usuário: Cadastro

2.4.2 Atualização de Usuário — PUT /usuarios/id

Tabela 5: Sequência: Atualização de Usuário

	Origem	Destino	Mensagem
1	Cliente HTTP	UsuarioController	PUT /usuarios/id
2	UsuarioController	UsuarioService	update(id, dto)
3	UsuarioService	UsuarioRepository	findById(id)
3.1a	UsuarioRepository	UsuarioService	Optional.empty() → 404 NOT FOUND
4	UsuarioService	UsuarioRepository	save(usuario)
4.1	UsuarioRepository	UsuarioService	usuario
5	UsuarioController	Cliente HTTP	UsuarioResponseDTO / 200 OK

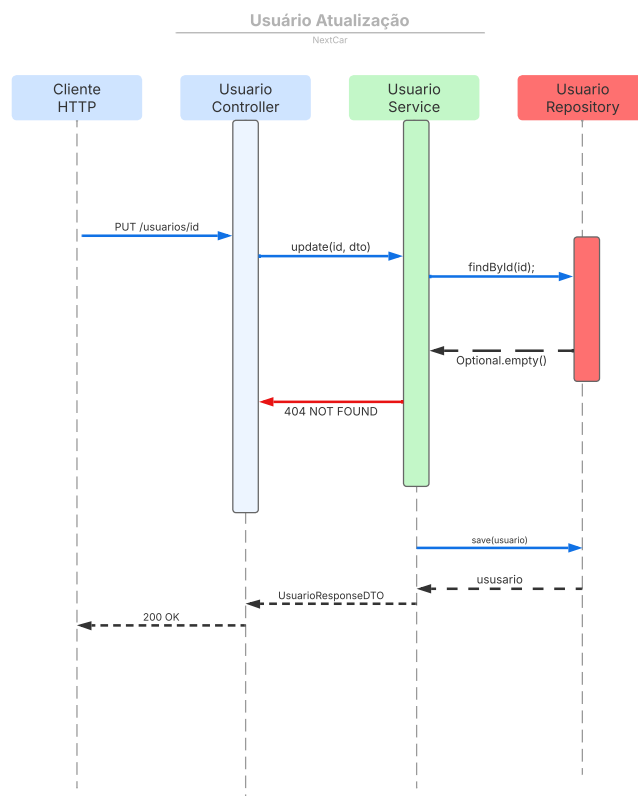


Figura 4: Diagrama de Sequência — Usuário: Atualização

2.4.3 Consultar Usuários — GET /usuarios

Tabela 6: Sequência: Consultar Todos os Usuários

	Origem	Destino	Mensagem
1	Cliente HTTP	UsuarioController	GET /usuarios
2	UsuarioController	UsuarioService	findAll()
3	UsuarioService	UsuarioRepository	findAll()
4	UsuarioRepository	UsuarioService	List<Usuario>
5	UsuarioController	Cliente HTTP	UsuarioResponseDTO / 200 OK

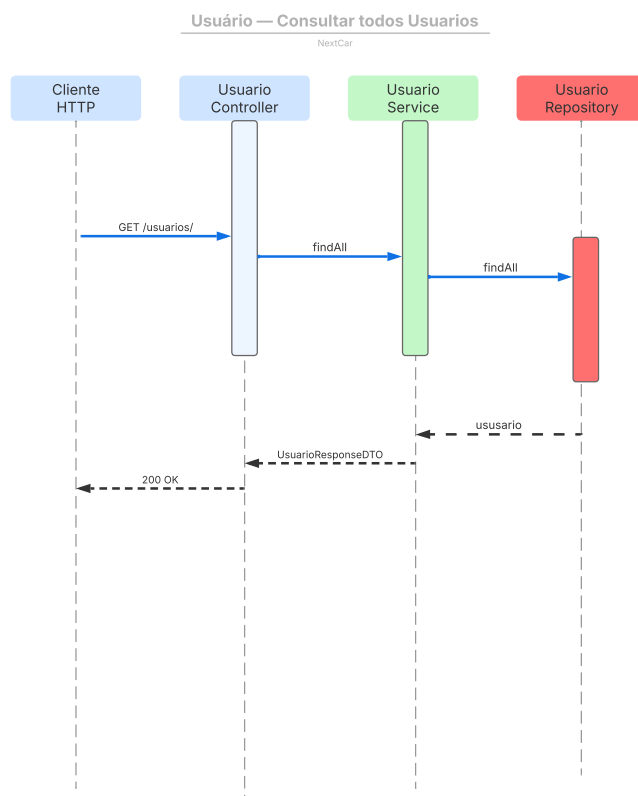


Figura 5: Diagrama de Sequência — Usuário: Consultar todos

2.4.4 Remoção de Usuário — DELETE /usuarios/id

Tabela 7: Sequência: Remoção de Usuário

	Origem	Destino	Mensagem
1	Cliente HTTP	UsuarioController	DELETE /usuarios/id
2	UsuarioController	UsuarioService	delete(id)
3	UsuarioService	UsuarioRepository	findById(id)
4	UsuarioService	UsuarioRepository	delete(usuario) «destroy»
5	UsuarioController	Cliente HTTP	204 NO CONTENT

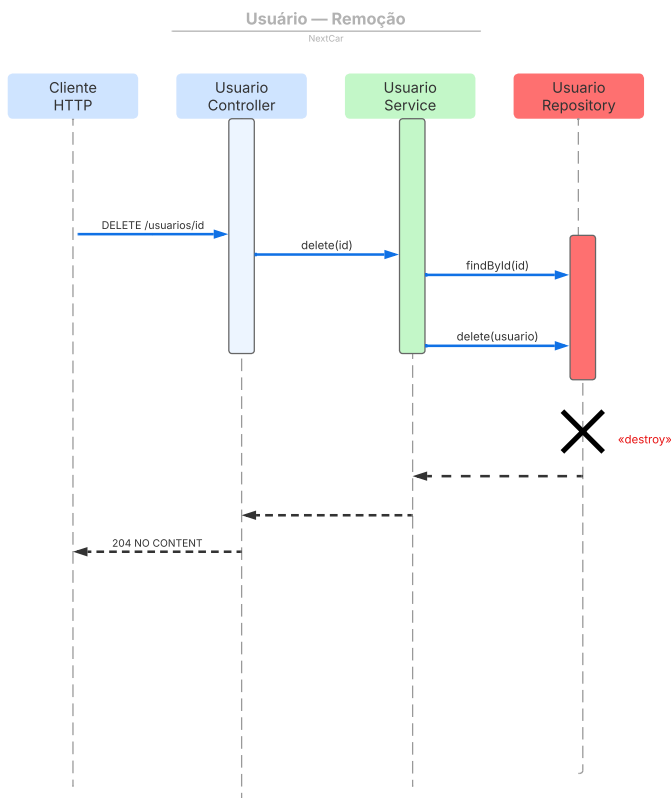


Figura 6: Diagrama de Sequência — Usuário: Remoção

2.5 Diagramas de Sequência — Veículo

Objetivo dos Diagramas

Detalhar as trocas de mensagens do módulo de **Veículo**, incluindo o fluxo de precificação via Strategy e Factory.

2.5.1 Cadastro de Veículo — POST /veiculos

Tabela 8: Sequência: Cadastro de Veículo

	Origem	Destino	Mensagem
1	Cliente HTTP	VeiculoController	POST /veiculos
2	VeiculoController	VeiculoService	save()
3	VeiculoService	VeiculoRepository	save()
3.1	VeiculoRepository	VeiculoService	Veiculo persistido
4	VeiculoController	Cliente HTTP	201 CREATED

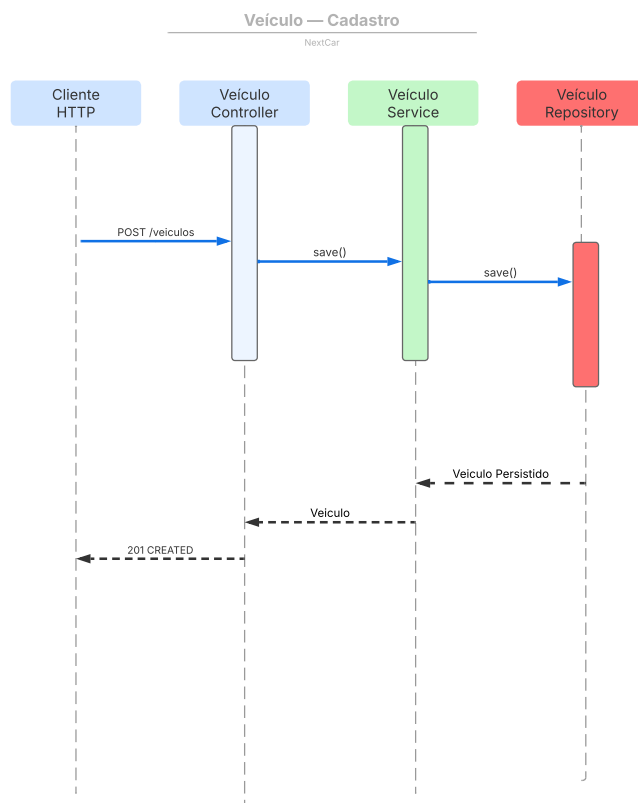


Figura 7: Diagrama de Sequência — Veículo: Cadastro

2.5.2 Atualização de Veículo — PUT /veiculos/id

Tabela 9: Sequência: Atualização de Veículo

	Origem	Destino	Mensagem
1	Cliente HTTP	VeiculoController	PUT /veiculos/id
2	VeiculoController	VeiculoService	update(veiculo/id)
3	VeiculoService	PrecoStrategyFactory	getStrategy(tipoPreco)
3.1a	PrecoStrategyFac	VeiculoService	TipoPrecoInvalidoException → 400 BAD REQUEST
3.1b	PrecoStrategyFac	VeiculoService	PrecoStrategy (válida)
4	VeiculoService	PrecoStrategy	calcularPreco()
5	VeiculoService	VeiculoRepository	save(veiculo)
6	VeiculoController	Cliente HTTP	200 OK

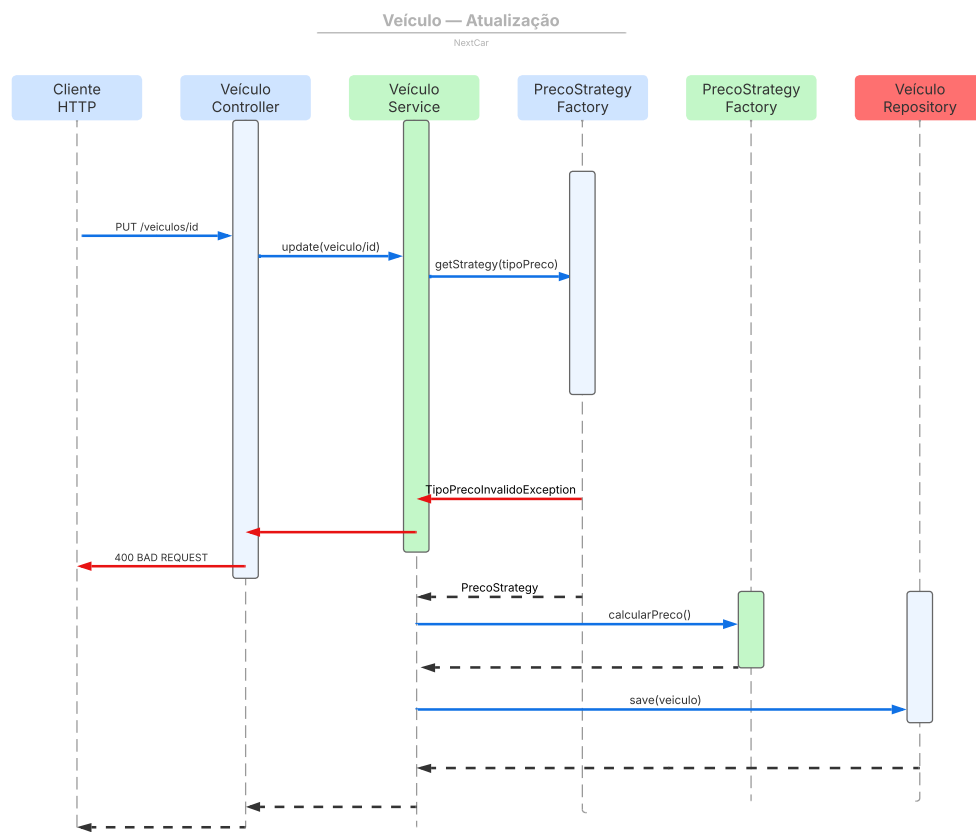


Figura 8: Diagrama de Sequência — Veículo: Atualização

2.5.3 Consultar Veículos — GET /veiculos

Tabela 10: Sequência: Consultar Todos os Veículos

	Origem	Destino	Mensagem
1	Cliente HTTP	VeiculoController	GET /veiculos?marca=X
2	VeiculoController	VeiculoService	findAll()
3	VeiculoService	VeiculoRepository	findAll()
4	VeiculoRepository	VeiculoService	List<Veiculo>
5	VeiculoController	Cliente HTTP	200 OK

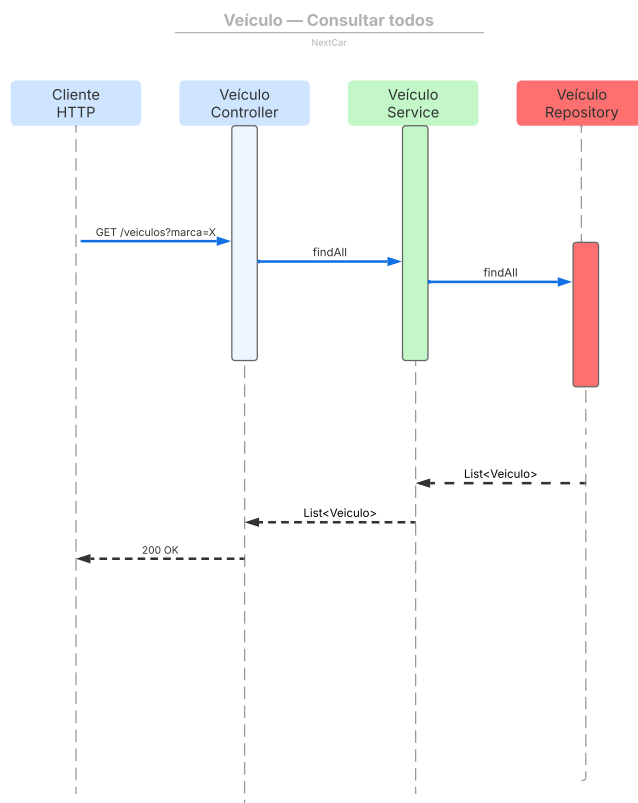


Figura 9: Diagrama de Sequência — Veículo: Consultar todos

2.5.4 Remoção de Veículo — DELETE /veiculos/id

Tabela 11: Sequência: Remoção de Veículo

	Origem	Destino	Mensagem
1	Cliente HTTP	VeiculoController	DELETE /veiculos/id
2	VeiculoController	VeiculoService	delete(id)
3	VeiculoService	VeiculoRepository	findById(id)
4	VeiculoService	VeiculoRepository	delete(veiculo) «destroy»
5	VeiculoController	Cliente HTTP	204 NO CONTENT

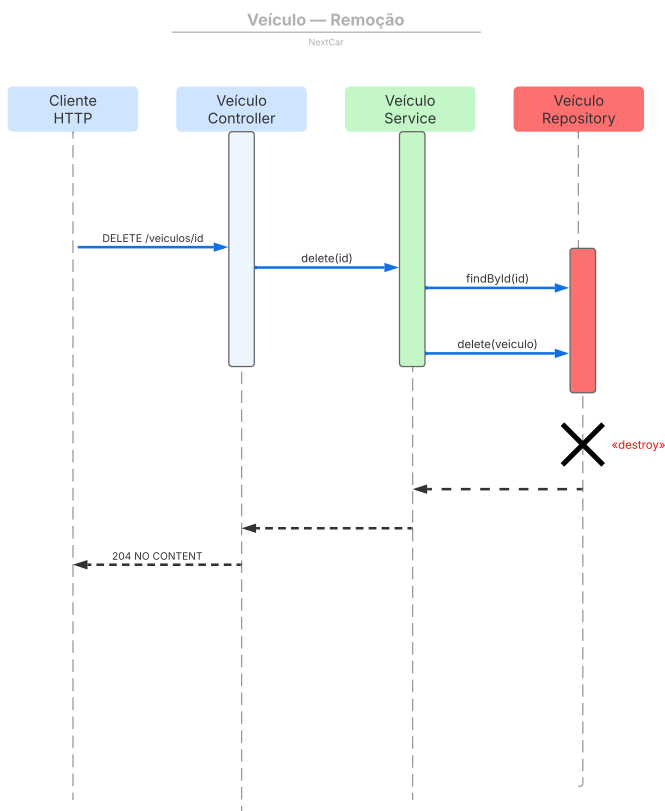


Figura 10: Diagrama de Sequência — Veículo: Remoção

2.6 Diagramas de Sequência — Venda

Objetivo dos Diagramas

Detalhar as trocas de mensagens do módulo de **Venda**, cobrindo processamento, atualização de status, consulta e remoção.

2.6.1 Processamento de Venda — POST /vendas

Tabela 12: Sequência: Processamento de Venda

	Origem	Destino	Mensagem
1	Cliente HTTP	VendaController	POST /vendas
2	VendaController	VendaService	efetuarVenda(dto) — valida veículo e usuário
3	VendaService	VendaRepository	save(venda)
3.1	VendaRepository	VendaService	Venda Salva
4	VendaController	VendaService	VendaResponseDTO
5	VendaController	Cliente HTTP	201 CREATED

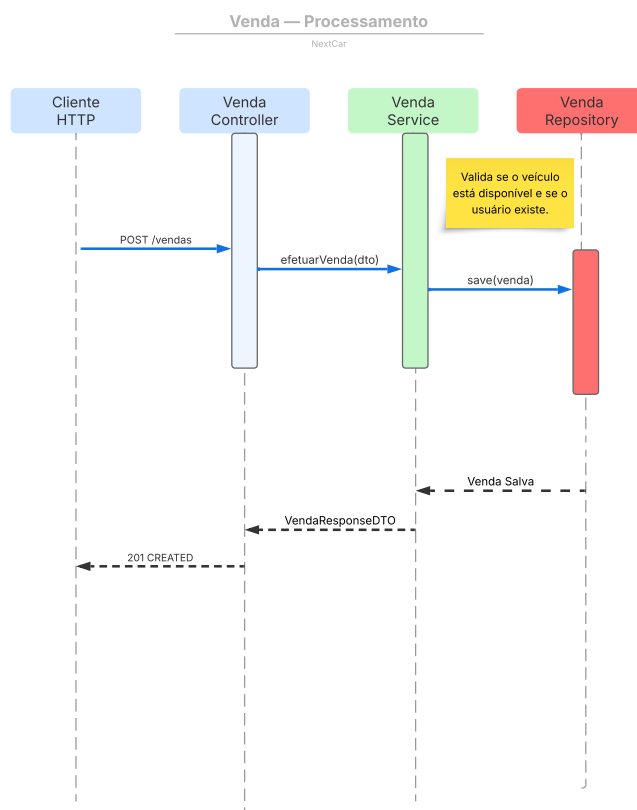


Figura 11: Diagrama de Sequência — Venda: Processamento

2.6.2 Atualização de Status de Venda — PUT /vendas/id

Tabela 13: Sequência: Atualização de Status de Venda

	Origem	Destino	Mensagem
1	Cliente HTTP	VendaController	PUT /vendas/id
2	VendaController	VendaService	atualizarStatus(id, status)
3	VendaService	VendaRepository	findById(id)
4	VendaService	VendaRepository	save(venda)
4.1	VendaRepository	VendaService	venda
5	VendaController	VendaService	VendaResponseDTO
6	VendaController	Cliente HTTP	200 OK

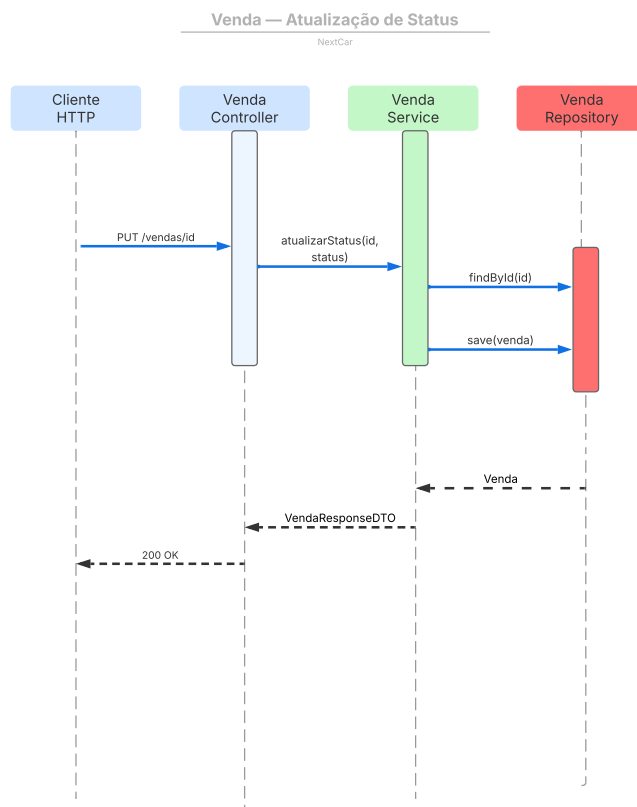


Figura 12: Diagrama de Sequência — Venda: Atualização de Status

2.6.3 Consultar Vendas — GET /vendas

Tabela 14: Sequência: Consultar Todas as Vendas

	Origem	Destino	Mensagem
1	Cliente HTTP	VendaController	GET /vendas
2	VendaController	VendaService	findAll()
3	VendaService	VendaRepository	findAll()
4	VendaRepository	VendaService	Venda
5	VendaController	Cliente HTTP	VendaResponseDTO / 200 OK

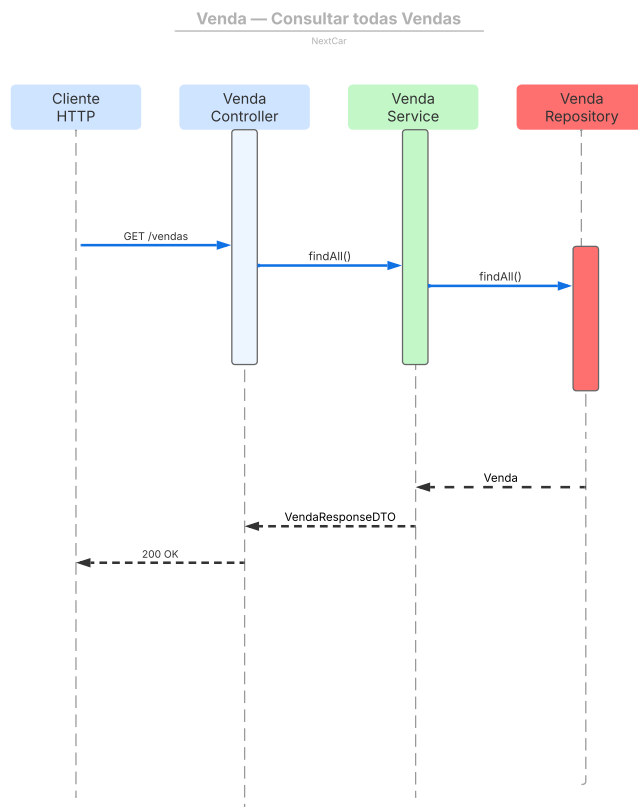


Figura 13: Diagrama de Sequência — Venda: Consultar todas

2.6.4 Remoção de Venda — DELETE /vendas/id

Tabela 15: Sequência: Remoção de Venda com Fallback Condicional

	Origem	Destino	Mensagem
1	Cliente HTTP	VendaController	DELETE /vendas/id
2	VendaController	VendaService	delete(id)
3	VendaService	VendaRepository	findById(id)
3.1a	VendaRepository	VendaService	Optional.empty() → Exception → 404 NOT FOUND
4	VendaService	VendaRepository	[venda encontrada] delete(venda) «destroy»
5	VendaController	Cliente HTTP	204 NO CONTENT

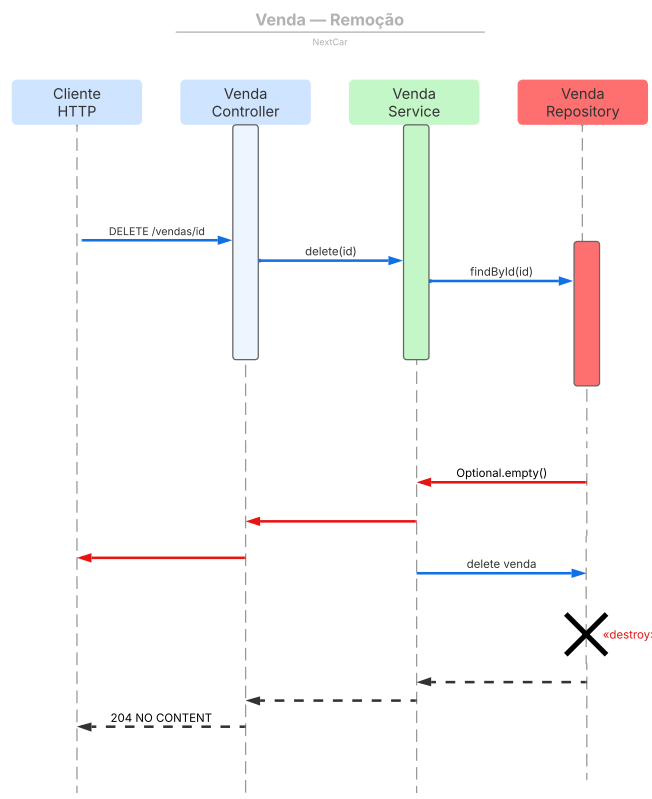


Figura 14: Diagrama de Sequência — Venda: Remoção

2.7 Diagramas de Colaboração — Veículo

Objetivo dos Diagramas

Mostrar como os objetos do módulo **Veículo** colaboram entre si para executar cada operação, destacando numeração de mensagens e dependências.

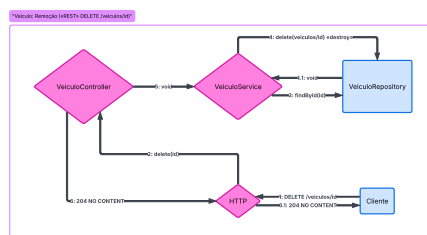
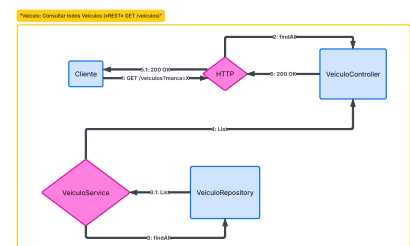
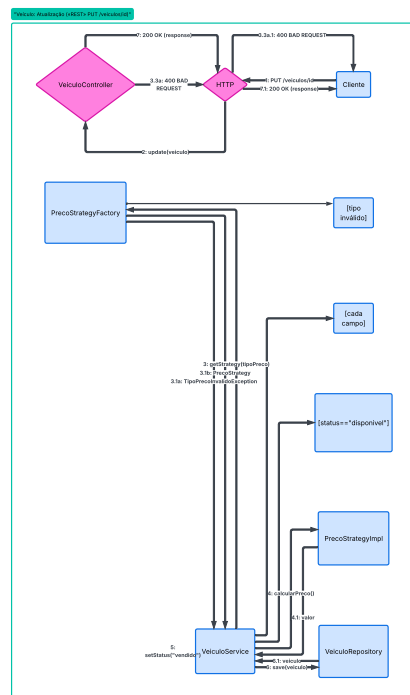
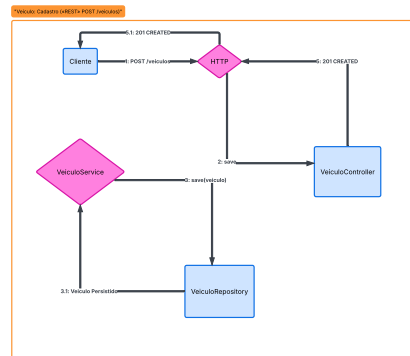


Figura 15: Diagrama de Colaboração — Veículo (Cadastro, Atualização, Consulta e Remoção)

Tabela 16: Colaboração — Módulo Veículo

Objeto	Colabora com	Papel
Cliente HTTP	VeiculoController	Origina todas as requisições REST relacionadas ao módulo de veículos.
VeiculoController	VeiculoService	Delega a lógica de negócio para a camada de serviço.
VeiculoService	PrecoStrategyFactory	Solicita a estratégia de precificação antes da persistência do veículo.
PrecoStrategyFactory	PrecoStrategyImpl	Retorna a implementação concreta da estratégia ou lança exceção quando necessário.
VeiculoService	VeiculoRepository	Persiste, consulta e remove informações de veículos no banco de dados.

2.8 Diagramas de Colaboração — Venda

Objetivo dos Diagramas

Mostrar como os objetos do módulo **Venda** colaboram para processar, atualizar, consultar e remover vendas.

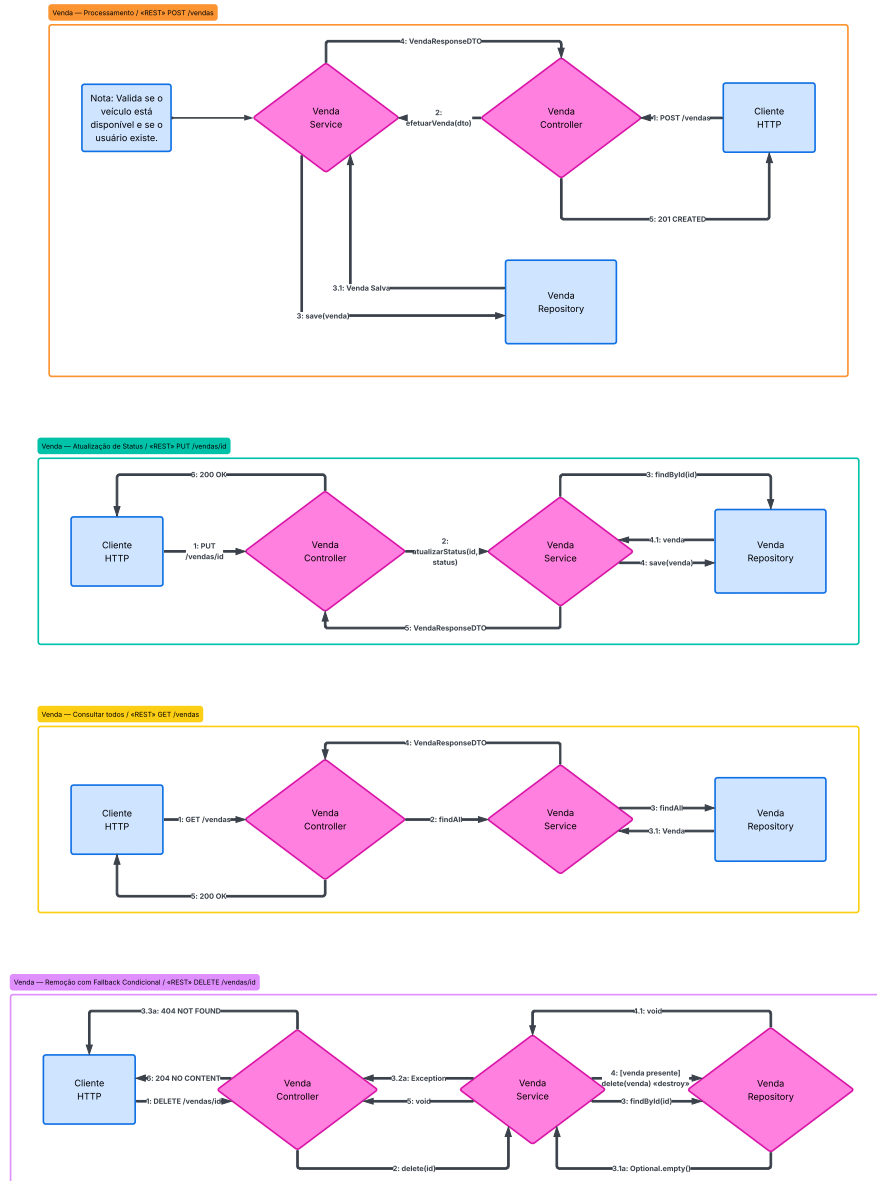


Figura 16: Diagrama de Colaboração — Venda (Processamento, Atualização, Consulta e Remoção)

Tabela 17: Colaboração — Módulo Venda

Objeto	Colabora com	Papel
Cliente HTTP	VendaController	Origina todas as requisições REST relacionadas ao módulo de vendas.
VendaController	VendaService	Delega o processamento das requisições e validações para a camada de serviço.
VendaService	VendaRepository	Persiste, consulta e remove registros de vendas no banco de dados.
VendaService	(Regra interna)	Valida a disponibilidade do veículo e a existência do usuário antes da conclusão da venda.

2.9 Diagramas de Colaboração — Usuário

Objetivo dos Diagramas

Mostrar como os objetos do módulo **Usuário** colaboram para cadastrar, atualizar, consultar e remover usuários.

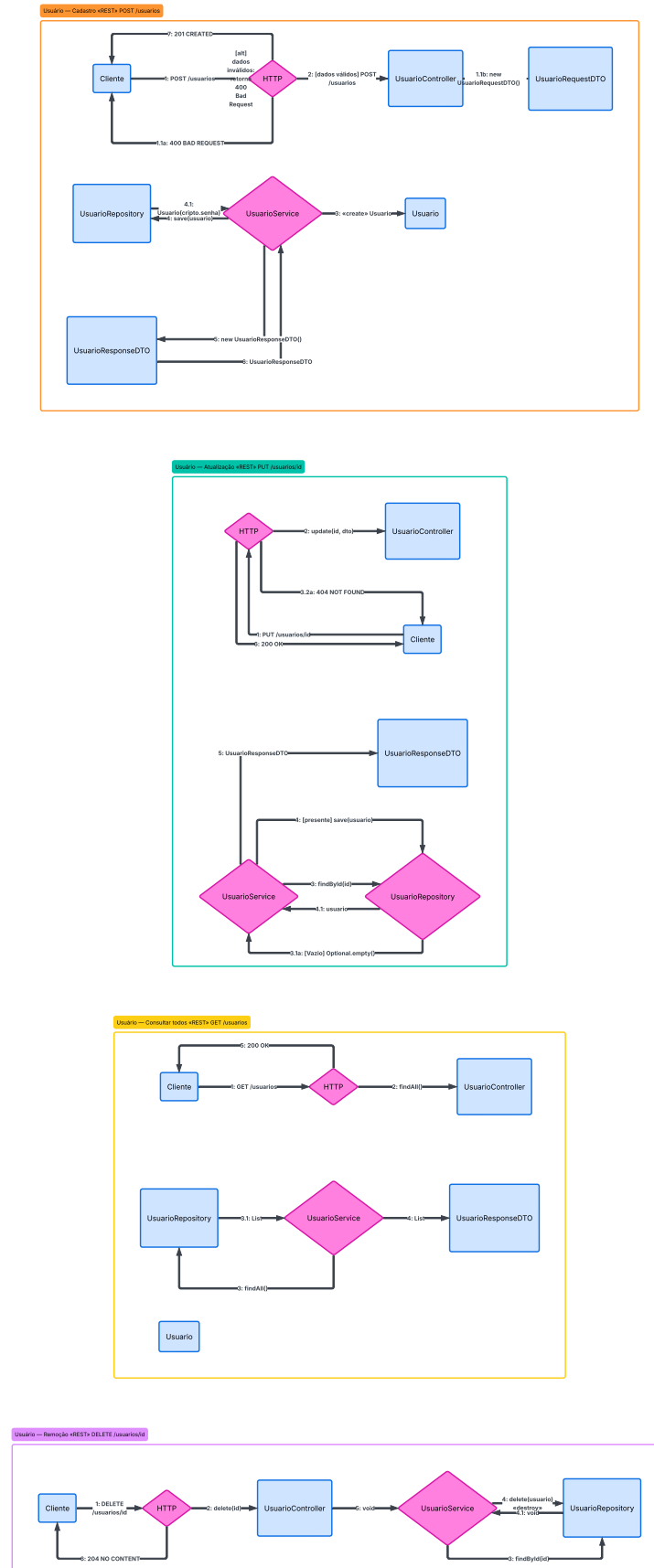


Figura 17: Diagrama de Colaboração — Usuário (Cadastro, Atualização, Consulta e Remoção)

Tabela 18: Colaboração — Módulo Usuário

Objeto	Colabora com	Papel
Cliente HTTP	UsuarioController	Origina todas as requisições REST relacionadas ao módulo de usuário.
UsuarioController	UsuarioService	Delega validações e regras de negócio da aplicação.
UsuarioService	UsuarioRequestDTO	Utiliza o DTO de entrada para encapsular os dados recebidos pela API.
UsuarioService	UsuarioRepository	Persiste, consulta e remove informações de usuários no banco de dados.
UsuarioService	UsuarioResponseDTO	Constrói o objeto de resposta antes de retornar ao controller.

3 Memorial Descritivo de Padrões e Princípios

3.1 Tabela de Padrões GoF

Tabela 19: Padrões GoF Aplicados no NextCar

Padrão	Classe/Módulo	Problema Resolvido	Justificativa
Strategy	PrecoStrategy	Variação de algoritmos de precificação.	Permite alternar entre preço normal, desconto e premium sem alterar o fluxo principal de venda.
Factory Method	PrecoStrategyFactory	Criação e seleção de estratégias concretas.	Centraliza a escolha da estratégia e reduz condicionais espalhadas pelo sistema.
Facade	VendaFacade	Complexidade do processo de venda.	Fornece uma interface única para os controllers e coordena múltiplos serviços internos.

3.2 Análise SOLID/GRASP

Tabela 20: *Princípios Aplicados no Projeto*

Princípio	Aplicação no NextCar	Benefício Técnico
SRP	Services, controllers e repositories possuem responsabilidades separadas.	Reduz o impacto de mudanças e melhora a organização do código.
OCP	Novas estratégias de preço podem ser adicionadas sem alterar o fluxo principal.	Facilita a evolução e extensão do sistema.
LSP	Estratégias concretas substituem <code>PrecoStrategy</code> .	Garante polimorfismo seguro entre implementações.
ISP	A interface <code>PrecoStrategy</code> é pequena e objetiva.	Evita dependências desnecessárias entre componentes.
DIP	As classes dependem de serviços e abstrações injetadas.	Aumenta a testabilidade e reduz o acoplamento.
GRASP Controller	Os controllers recebem os eventos e requisições do sistema.	Organiza a entrada da aplicação de forma centralizada.
GRASP Expert	Os services concentram regras relacionadas às suas entidades.	Melhora a coesão das responsabilidades.
Low Coupling	DTOs, Facade e Repository reduzem dependências diretas.	Facilita manutenção e evolução do software.
High Cohesion	Cada classe possui uma função bem definida.	Melhora compreensão, reutilização e testes.

4 Arquitetura e Qualidade

4.1 Descrição Arquitetural

O sistema segue uma arquitetura em camadas. Cada camada possui uma responsabilidade específica, reduzindo o acoplamento e aumentando a manutenibilidade da aplicação.

Tabela 21: Arquitetura em Camadas

Camada	Pacote	Responsabilidade
Apresentação	<code>nextcar.controller</code>	Recebe requisições HTTP e retorna respostas REST.
Transporte	<code>nextcar.dto</code>	Define os objetos de entrada e saída da API.
Negócio	<code>nextcar.service</code>	Executa regras de negócio e orquestra os casos de uso.
Domínio	<code>nextcar.model</code>	Representa as entidades persistentes do sistema.
Persistência	<code>nextcar.repository</code>	Realiza o acesso ao banco de dados utilizando Spring Data JPA.
Infraestrutura	<code>nextcar.configuration</code>	Centraliza configurações e mecanismos de segurança da API.
Exceções	<code>nextcar.exception</code>	Centraliza o tratamento de exceções da aplicação.

4.2 Relatório de Métricas

Tabela 22: Complexidade Ciclomática

Pacote	Total	Perdida	Coberta
<code>nextcar.service</code>	34	0	34 (100%)
<code>nextcar.controller</code>	15	9	6 (40%)
<code>nextcar.model</code>	6	0	6 (100%)
Total	71	13	58 (82%)

Tabela 23: Cobertura de Código JaCoCo

Pacote	Instruções	Branches	Linhas	Leitura Técnica
nextcar.service	100%	100%	88/88	Camada de negócio amplamente validada pelos testes.
nextcar.model	100%	n/a	24/24	Entidades completamente cobertas pelos testes.
Total	90%	100%	155/172	Resultado consistente, com possibilidade de evolução nos controllers.

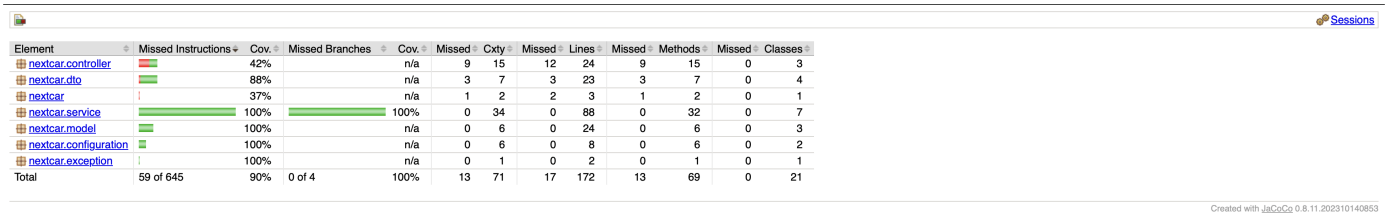
Tabela 24: Acoplamento entre Classes

Classe	Ca	Ce	I	Interpretação
VendaController	0	3	1.0	Classe instável por atuar como ponto de entrada HTTP da aplicação.
VendaFacade	1	4	0.8	Orquestrador central com dependências justificadas pela responsabilidade da camada.
PrecoStrategy	3	0	0.0	Abstração estável e reutilizável, com baixo acoplamento externo.

4.3 Plano de Testes

Tabela 25: Plano de Testes Unitários

Classe Testada	Ferramenta	Casos	Objetivo
UsuarioServiceTest	JUnit 5 + Mockito	7	Validar as regras de negócio relacionadas aos usuários.
VeiculoServiceTest	JUnit 5 + Mockito	8	Validar consultas e atualizações de veículos.
VendaFacadeTest	JUnit 5 + Mockito	6	Validar o fluxo completo do processo de venda.
Total	JaCoCo	32	Garantir a cobertura da camada de negócio da aplicação.



Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Ctxy	Missed Lines	Missed Methods	Missed Classes
nextcar.controller	42%	n/a	9	15	12	24	9	15
nextcar.dto	88%	n/a	3	7	3	23	3	7
nextcar	37%	n/a	1	2	2	3	1	2
nextcar.service	100%	100%	0	34	0	88	0	32
nextcar.model	100%	n/a	0	6	0	24	0	6
nextcar.configuration	100%	n/a	0	6	0	8	0	6
nextcar.exception	100%	n/a	0	1	0	2	0	1
Total	59 of 645	90%	0 of 4	100%	13	71	17	172

Figura 18: Print do JaCoCo - Sistema NextCar

4.3.1 Scripts de Testes Unitários

Para garantir o isolamento das regras de negócio e validar de forma cirúrgica o comportamento de fábricas, serviços e fachadas, foram desenvolvidos scripts com **JUnit 5** e **Mockito**. O uso de mocks anula a dependência de infraestrutura (banco de dados), focando estritamente na coerência lógica do design.

```

1 package nextcar.service;
2
3 import nextcar.exception.TipoPrecoInvalidoException;
4 import org.junit.jupiter.api.BeforeEach;
5 import org.junit.jupiter.api.Test;
6 import java.util.Map;
7 import static org.junit.jupiter.api.Assertions.*;
8
9 class PrecoStrategyFactoryTest {
10

```



```
11     private PrecoStrategyFactory factory;
12
13     @BeforeEach
14     void setUp() {
15         factory = new PrecoStrategyFactory(Map.of(
16             "normal", new PrecoNormal(),
17             "desconto", new PrecoDesconto()
18         ));
19     }
20
21     @Test
22     void testGetStrategyNormal() {
23         PrecoStrategy strategy = factory.getStrategy("normal");
24         assertNotNull(strategy);
25         assertEquals(100.0, strategy.calcularPreco(100.0));
26     }
27
28     @Test
29     void testGetStrategyDesconto() {
30         PrecoStrategy strategy = factory.getStrategy("desconto");
31         assertNotNull(strategy);
32         assertEquals(90.0, strategy.calcularPreco(100.0));
33     }
34
35     @Test
36     void testGetStrategyTipoInvalido() {
37         assertThrows(TipoPrecoInvalidoException.class,
38             () -> factory.getStrategy("invalido"));
39     }
40
41     @Test
42     void testMensagemExcecaoContemTipo() {
43         TipoPrecoInvalidoException ex = assertThrows(
44             TipoPrecoInvalidoException.class,
45             () -> factory.getStrategy("xyz"));
46         assertTrue(ex.getMessage().contains("xyz"));
47     }
48 }
```

Listing 1: Testes Unitários — Fábrica de Estratégias de Preço

```
1 package nextcar.service;
2
```

```
3 import jakarta.persistence.EntityNotFoundException;
4 import nextcar.dto.UsuarioRequestDTO;
5 import nextcar.dto.UsuarioResponseDTO;
6 import nextcar.model.Usuario;
7 import nextcar.repository.UsuarioRepository;
8 import org.junit.jupiter.api.BeforeEach;
9 import org.junit.jupiter.api.Test;
10 import org.junit.jupiter.api.extension.ExtendWith;
11 import org.mockito.InjectMocks;
12 import org.mockito.Mock;
13 import org.mockito.junit.jupiter.MockitoExtension;
14 import org.springframework.security.crypto.password.PasswordEncoder;
15
16 import java.util.List;
17 import java.util.Optional;
18
19 import static org.junit.jupiter.api.Assertions.*;
20 import static org.mockito.Mockito.*;
21
22 @ExtendWith(MockitoExtension.class)
23 class UsuarioServiceTest {
24
25     @Mock
26     private UsuarioRepository usuarioRepository;
27
28     @Mock
29     private PasswordEncoder encoder;
30
31     @InjectMocks
32     private UsuarioService usuarioService;
33
34     private Usuario usuario;
35     private UsuarioRequestDTO requestDTO;
36
37     @BeforeEach
38     void setUp() {
39         usuario = new Usuario();
40         usuario.setId(1L);
41         usuario.setNome("Emerson");
42         usuario.setLogin("emerson@test.com");
43         usuario.setSenha("senhaCriptografada");
44     }
```

```
45     requestDTO = new UsuarioRequestDTO("Emerson", "emerson@test.
46         com", "123");
47
48     @Test
49     void testSaveUsuario() {
50         when(encoder.encode("123")).thenReturn("senhaCriptografada");
51         when(usuarioRepository.save(any(Usuario.class))).thenReturn(
52             usuario);
53
54         UsuarioResponseDTO resultado = usuarioService.save(requestDTO
55             );
56
57         assertNotNull(resultado);
58         assertEquals("Emerson", resultado.getNome());
59         assertEquals("emerson@test.com", resultado.getLogin());
60         verify(encoder).encode("123");
61         verify(usuarioRepository).save(any(Usuario.class));
62     }
63
64     @Test
65     void testFindUsuarios() {
66         when(usuarioRepository.findAll()).thenReturn(List.of(usuario)
67             );
68
69         List<UsuarioResponseDTO> usuarios = usuarioService.find();
70
71         assertEquals(1, usuarios.size());
72         assertEquals("Emerson", usuarios.get(0).getNome());
73         verify(usuarioRepository).findAll();
74     }
75
76     @Test
77     void testUpdateUsuario() {
78         UsuarioRequestDTO novoDTO = new UsuarioRequestDTO("Novo Nome"
79             , "novo@test.com", "456");
80
81         when(usuarioRepository.findById(1L)).thenReturn(Optional.of(
82             usuario));
83         when(encoder.encode("456")).thenReturn("
84             novaSenhaCriptografada");
85         when(usuarioRepository.save(any(Usuario.class))).thenReturn(
```

```
        usuario);
80
81        UsuarioResponseDTO resultado = usuarioService.update(novoDTO,
            1L);
82
83        assertNotNull(resultado);
84        verify(usuarioRepository).findById(1L);
85        verify(encoder).encode("456");
86        verify(usuarioRepository).save(usuario);
87    }
88
89    @Test
90    void testUpdateUsuarioNotFound() {
91        when(usuarioRepository.findById(1L)).thenReturn(Optional.
            empty());
92
93        assertThrows(EntityNotFoundException.class,
94            () -> usuarioService.update(requestDTO, 1L));
95
96        verify(usuarioRepository).findById(1L);
97    }
98
99    @Test
100    void testDeleteUsuario() {
101        when(usuarioRepository.findById(1L)).thenReturn(Optional.of(
            usuario));
102
103        usuarioService.delete(1L);
104
105        verify(usuarioRepository).findById(1L);
106        verify(usuarioRepository).delete(usuario);
107    }
108
109    @Test
110    void testDeleteUsuarioNotFound() {
111        when(usuarioRepository.findById(1L)).thenReturn(Optional.
            empty());
112
113        assertThrows(EntityNotFoundException.class,
114            () -> usuarioService.delete(1L));
115
116        verify(usuarioRepository).findById(1L);
```

```
117     }
118
119     @Test
120     void testBuscarPorIdNotFound() {
121         when(usuarioRepository.findById(99L)).thenReturn(Optional.
122             empty());
123
124         assertThrows(EntityNotFoundException.class,
125             () -> usuarioService.buscarPorId(99L));
126
127         verify(usuarioRepository).findById(99L);
128     }
129 }
```

Listing 2: Testes Unitários — Serviço de Usuários com Mocks

```
1 package nextcar.service;
2
3 import jakarta.persistence.EntityNotFoundException;
4 import nextcar.exception.TipoPrecoInvalidoException;
5 import nextcar.model.Veiculo;
6 import nextcar.repository.VeiculoRepository;
7 import org.junit.jupiter.api.BeforeEach;
8 import org.junit.jupiter.api.Test;
9 import org.junit.jupiter.api.extension.ExtendWith;
10 import org.mockito.InjectMocks;
11 import org.mockito.Mock;
12 import org.mockito.junit.jupiter.MockitoExtension;
13
14 import java.util.List;
15 import java.util.Optional;
16
17 import static org.junit.jupiter.api.Assertions.*;
18 import static org.mockito.Mockito.*;
19
20 @ExtendWith(MockitoExtension.class)
21 class VeiculoServiceTest {
22
23     @Mock
24     private VeiculoRepository veiculoRepository;
25
26     @Mock
27     private PrecoStrategyFactory strategyFactory;
```

```
28
29     @Mock
30     private PrecoStrategy precoStrategy;
31
32     @InjectMocks
33     private VeiculoService veiculoService;
34
35     private Veiculo veiculo;
36
37     @BeforeEach
38     void setUp() {
39         veiculo = new Veiculo();
40         veiculo.setId(1L);
41         veiculo.setMarca("Toyota");
42         veiculo.setModelo("Corolla");
43         veiculo.setAno(2020);
44         veiculo.setPreco(80000.0);
45         veiculo.setTipo("Sedan");
46         veiculo.setStatus("disponivel");
47         veiculo.setTipoPreco("normal");
48     }
49
50     @Test
51     void testSaveVeiculo() {
52         when(veiculoRepository.save(veiculo)).thenReturn(veiculo);
53
54         Veiculo resultado = veiculoService.save(veiculo);
55
56         assertNotNull(resultado);
57         assertEquals("Toyota", resultado.getMarca());
58         verify(veiculoRepository).save(veiculo);
59     }
60
61     @Test
62     void testFindVeiculos() {
63         when(veiculoRepository.findAll()).thenReturn(List.of(veiculo));
64
65         assertEquals(1, veiculoService.find().size());
66         verify(veiculoRepository).findAll();
67     }
68
```

```
69     @Test
70     void testUpdateVeiculo() {
71         Veiculo novo = new Veiculo();
72         novo.setMarca("Honda");
73         novo.setModelo("Civic");
74         novo.setAno(2022);
75         novo.setPreco(120000.0);
76         novo.setTipo("Sedan");
77         novo.setStatus("disponivel");
78         novo.setTipoPreco("normal");
79
80         when(veiculoRepository.findById(1L)).thenReturn(Optional.of(
81             veiculo));
82         when(veiculoRepository.save(any(Veiculo.class))).thenReturn(
83             veiculo);
84         when(strategyFactory.getStrategy("normal")).thenReturn(
85             precoStrategy);
86         when(precoStrategy.calcularPreco(120000.0)).thenReturn
87             (120000.0);
88
89         Veiculo resultado = veiculoService.update(novo, 1L);
90
91         assertNotNull(resultado);
92         verify(strategyFactory).getStrategy("normal");
93         verify(precoStrategy).calcularPreco(120000.0);
94         verify(veiculoRepository).save(veiculo);
95     }
96
97     @Test
98     void testUpdateVeiculoNotFound() {
99         when(veiculoRepository.findById(1L)).thenReturn(Optional.
100             empty());
101
102         assertThrows(EntityNotFoundException.class,
103             () -> veiculoService.update(veiculo, 1L));
104     }
105
106     @Test
107     void testUpdateVeiculoTipoPrecoInvalido() {
108         Veiculo novo = new Veiculo();
109         novo.setMarca("Honda");
110         novo.setModelo("Civic");
```

```
106     novo.setAno(2022);
107     novo.setPreco(100000.0);
108     novo.setTipo("Sedan");
109     novo.setStatus("disponivel");
110     novo.setTipoPreco("invalido");
111
112     when(veiculoRepository.findById(1L)).thenReturn(Optional.of(
113         veiculo));
114     when(strategyFactory.getStrategy("invalido"))
115         .thenThrow(new TipoPrecoInvalidoException("invalido"));
116
117     assertThrows(TipoPrecoInvalidoException.class,
118         () -> veiculoService.update(novo, 1L));
119
120     verify(strategyFactory).getStrategy("invalido");
121 }
122
123 @Test
124 void testDeleteVeiculo() {
125     when(veiculoRepository.findById(1L)).thenReturn(Optional.of(
126         veiculo));
127
128     veiculoService.delete(1L);
129
130     verify(veiculoRepository).delete(veiculo);
131 }
132
133 @Test
134 void testDeleteVeiculoNotFound() {
135     when(veiculoRepository.findById(1L)).thenReturn(Optional.
136         empty());
137
138     assertThrows(EntityNotFoundException.class,
139         () -> veiculoService.delete(1L));
140 }
141
142 @Test
143 void testBuscarPorIdNotFound() {
144     when(veiculoRepository.findById(99L)).thenReturn(Optional.
145         empty());
```



```
143         assertThrows(EntityNotFoundException.class,
144             () -> veiculoService.buscarPorId(99L));
145     }
146 }
```

Listing 3: Testes Unitários — Serviço de Veículos e Acoplamento de Regras

```
1 package nextcar.service;
2
3 import nextcar.dto.VendaRequestDTO;
4 import nextcar.dto.VendaResponseDTO;
5 import nextcar.model.Usuario;
6 import nextcar.model.Veiculo;
7 import nextcar.model.Venda;
8 import org.junit.jupiter.api.BeforeEach;
9 import org.junit.jupiter.api.Test;
10 import org.junit.jupiter.api.extension.ExtendWith;
11 import org.mockito.InjectMocks;
12 import org.mockito.Mock;
13 import org.mockito.junit.jupiter.MockitoExtension;
14
15 import java.time.LocalDate;
16 import java.util.List;
17
18 import static org.junit.jupiter.api.Assertions.*;
19 import static org.mockito.Mockito.*;
20
21 @ExtendWith(MockitoExtension.class)
22 class VendaFacadeTest {
23
24     @Mock private VendaService vendaService;
25     @Mock private VeiculoService veiculoService;
26     @Mock private UsuarioService usuarioService;
27     @Mock private PrecoStrategyFactory strategyFactory;
28     @Mock private PrecoStrategy precoStrategy;
29
30     @InjectMocks
31     private VendaFacade facade;
32
33     private Veiculo veiculo;
34     private Usuario usuario;
35     private Venda venda;
36 }
```

```
37     @BeforeEach
38     void setUp() {
39         veiculo = new Veiculo();
40         veiculo.setId(1L);
41         veiculo.setMarca("Toyota");
42         veiculo.setModelo("Corolla");
43         veiculo.setTipoPreco("normal");
44         veiculo.setPreco(80000.0);
45         veiculo.setStatus("disponivel");
46
47         usuario = new Usuario();
48         usuario.setId(1L);
49         usuario.setNome("Emerson");
50         usuario.setLogin("emerson@test.com");
51         usuario.setSenha("hash");
52
53         venda = new Venda(LocalDate.now(), 80000.0, veiculo, usuario)
54             ;
55         venda.setId(1L);
56     }
57
58     @Test
59     void testRegistrarVenda() {
60         VendaRequestDTO dto = new VendaRequestDTO();
61         dto.setData(LocalDate.now());
62         dto.setValorFinal(80000.0);
63         dto.setVeiculoId(1L);
64         dto.setUsuarioId(1L);
65
66         when(veiculoService.buscarPorId(1L)).thenReturn(veiculo);
67         when(usuarioService.buscarPorId(1L)).thenReturn(usuario);
68         when(strategyFactory.getStrategy("normal")).thenReturn(
69             precoStrategy);
70         when(precoStrategy.calcularPreco(80000.0)).thenReturn(
71             80000.0);
72         when(vendaService.save(any(Venda.class))).thenReturn(venda);
73
74         VendaResponseDTO response = facade.registrarVenda(dto);
75
76         assertNotNull(response);
77         assertEquals(80000.0, response.getValorFinal());
78         assertEquals("Toyota Corolla", response.getVeiculoDescricao())
```

```
        );
76     assertEquals("Emerson", response.getUsuarioNome());
77     verify(strategyFactory).getStrategy("normal");
78     verify(precoStrategy).calcularPreco(80000.0);
79     verify(veiculoService).buscarPorId(1L);
80     verify(usuarioService).buscarPorId(1L);
81     verify(vendaService).save(any(Venda.class));
82 }
83
84 @Test
85 void testRegistrarVendaMudaStatusParaVendido() {
86     VendaRequestDTO dto = new VendaRequestDTO();
87     dto.setData(LocalDate.now());
88     dto.setValorFinal(80000.0);
89     dto.setVeiculoId(1L);
90     dto.setUsuarioId(1L);
91
92     when(veiculoService.buscarPorId(1L)).thenReturn(veiculo);
93     when(usuarioService.buscarPorId(1L)).thenReturn(usuario);
94     when(strategyFactory.getStrategy("normal")).thenReturn(
95         precoStrategy);
96     when(precoStrategy.calcularPreco(80000.0)).thenReturn(
97         80000.0);
98     when(vendaService.save(any(Venda.class))).thenReturn(venda);
99
100    facade.registrarVenda(dto);
101
102    assertEquals("vendido", veiculo.getStatus());
103    verify(veiculoService).save(veiculo);
104 }
105
106 @Test
107 void testRegistrarVendaVeiculoJaVendido() {
108     veiculo.setStatus("vendido");
109
110     VendaRequestDTO dto = new VendaRequestDTO();
111     dto.setData(LocalDate.now());
112     dto.setValorFinal(80000.0);
113     dto.setVeiculoId(1L);
114     dto.setUsuarioId(1L);
115
116     when(veiculoService.buscarPorId(1L)).thenReturn(veiculo);
```

```
115         when(usuarioService.buscarPorId(1L)).thenReturn(usuario);
116         when(strategyFactory.getStrategy("normal")).thenReturn(
117             precoStrategy);
118         when(precoStrategy.calcularPreco(80000.0)).thenReturn
119             (80000.0);
120         when(vendaService.save(any(Venda.class))).thenReturn(venda);
121
122         facade.registrarVenda(dto);
123
124         assertEquals("vendido", veiculo.getStatus());
125         verify(veiculoService, never()).save(any());
126     }
127
128     @Test
129     void testListarVendas() {
130         when(vendaService.find()).thenReturn(List.of(venda));
131
132         List<VendaResponseDTO> lista = facade.listarVendas();
133
134         assertEquals(1, lista.size());
135         verify(vendaService).find();
136     }
137
138     @Test
139     void testAtualizarVenda() {
140         VendaRequestDTO dto = new VendaRequestDTO();
141         dto.setData(LocalDate.now());
142         dto.setValorFinal(95000.0);
143         dto.setVeiculoId(1L);
144
145         when(veiculoService.buscarPorId(1L)).thenReturn(veiculo);
146         when(vendaService.update(any(Venda.class), eq(1L))).
147             thenReturn(venda);
148
149         VendaResponseDTO response = facade.atualizarVenda(dto, 1L);
150
151         assertNotNull(response);
152         verify(vendaService).update(any(Venda.class), eq(1L));
153     }
154
155     @Test
156     void testCancelarVenda() {
```

```
154         facade.cancelarVenda(1L);
155         verify(vendaService).delete(1L);
156     }
157 }
```

Listing 4: Testes Unitários — Fachada de Vendas (VendaFacade)

```
1 package nextcar.service;
2
3 import jakarta.persistence.EntityNotFoundException;
4 import nextcar.model.Usuario;
5 import nextcar.model.Veiculo;
6 import nextcar.model.Venda;
7 import nextcar.repository.VendaRepository;
8 import org.junit.jupiter.api.BeforeEach;
9 import org.junit.jupiter.api.Test;
10 import org.junit.jupiter.api.extension.ExtendWith;
11 import org.mockito.InjectMocks;
12 import org.mockito.Mock;
13 import org.mockito.junit.jupiter.MockitoExtension;
14
15 import java.time.LocalDate;
16 import java.util.List;
17 import java.util.Optional;
18
19 import static org.junit.jupiter.api.Assertions.*;
20 import static org.mockito.Mockito.*;
21
22 @ExtendWith(MockitoExtension.class)
23 class VendaServiceTest {
24
25     @Mock
26     private VendaRepository vendaRepository;
27
28     @InjectMocks
29     private VendaService vendaService;
30
31     private Venda venda;
32     private Veiculo veiculo;
33     private Usuario usuario;
34
35     @BeforeEach
36     void setUp() {
```

```
37     veiculo = new Veiculo();
38     veiculo.setId(1L);
39     veiculo.setMarca("Toyota");
40     veiculo.setModelo("Corolla");
41     veiculo.setTipoPreco("normal");
42     veiculo.setStatus("disponivel");
43     veiculo.setPreco(80000.0);
44
45     usuario = new Usuario();
46     usuario.setId(1L);
47     usuario.setNome("Emerson");
48     usuario.setLogin("emerson@test.com");
49     usuario.setSenha("123");
50
51     venda = new Venda();
52     venda.setId(1L);
53     venda.setData(LocalDate.now());
54     venda.setValorFinal(80000.0);
55     venda.setVeiculo(veiculo);
56     venda.setUsuario(usuario);
57 }
58
59 @Test
60 void testSaveVenda() {
61     when(vendaRepository.save(venda)).thenReturn(venda);
62     Venda resultado = vendaService.save(venda);
63     assertNotNull(resultado);
64     assertEquals(80000.0, resultado.getValorFinal());
65     verify(vendaRepository).save(venda);
66 }
67
68 @Test
69 void testFindVendas() {
70     when(vendaRepository.findAll()).thenReturn(List.of(venda));
71     List<Venda> vendas = vendaService.find();
72     assertEquals(1, vendas.size());
73     verify(vendaRepository).findAll();
74 }
75
76 @Test
77 void testUpdateVenda() {
78     Venda vendaNova = new Venda();
```

```
79     vendaNova.setData(LocalDate.now());
80     vendaNova.setValorFinal(95000.0);
81     vendaNova.setVeiculo(veiculo);
82
83     when(vendaRepository.findById(1L)).thenReturn(Optional.of(
84         venda));
85
86     when(vendaRepository.save(any(Venda.class))).thenReturn(venda
87         );
88
89     Venda resultado = vendaService.update(vendaNova, 1L);
90
91     assertNotNull(resultado);
92     assertEquals(95000.0, resultado.getValorFinal());
93     verify(vendaRepository).findById(1L);
94     verify(vendaRepository).save(venda);
95 }
96
97 @Test
98 void testUpdateVendaNotFound() {
99     when(vendaRepository.findById(1L)).thenReturn(Optional.empty
100         ());
101     assertThrows(EntityNotFoundException.class, () ->
102         vendaService.update(venda, 1L));
103     verify(vendaRepository).findById(1L);
104 }
105
106 @Test
107 void testDeleteVenda() {
108     when(vendaRepository.findById(1L)).thenReturn(Optional.of(
109         venda));
110     vendaService.delete(1L);
111     verify(vendaRepository).findById(1L);
112     verify(vendaRepository).delete(venda);
113 }
114
115 @Test
116 void testDeleteVendaNotFound() {
117     when(vendaRepository.findById(1L)).thenReturn(Optional.empty
118         ());
119     assertThrows(EntityNotFoundException.class, () ->
120         vendaService.delete(1L));
121     verify(vendaRepository).findById(1L);
```

```
114     }
115
116     @Test
117     void testBuscarPorIdNotFound() {
118         when(vendaRepository.findById(99L)).thenReturn(Optional.empty());
119
120         assertThrows(EntityNotFoundException.class, () ->
121             vendaService.buscarPorId(99L));
122         verify(vendaRepository).findById(99L);
123     }
124 }
```

Listing 5: Testes Unitários — Serviço Transacional de Vendas

4.3.2 Scripts de Testes de Integração (End-to-End)

Com o intuito de simular o comportamento da aplicação em ambiente real, os testes de integração sobem o contexto completo do ecossistema Spring Boot mapeado a uma porta randômica segura. Utiliza-se o **WebTestClient** para realizar requisições HTTP e validar os contratos REST da API, realizando inserções estruturadas via JSONPath e garantindo a persistência correta nos repositórios.

```
1 package nextcar;
2
3 import nextcar.dto.UsuarioRequestDTO;
4 import nextcar.dto.VendaRequestDTO;
5 import nextcar.model.Veiculo;
6 import nextcar.repository.UsuarioRepository;
7 import nextcar.repository.VeiculoRepository;
8 import nextcar.repository.VendaRepository;
9 import org.junit.jupiter.api.BeforeEach;
10 import org.junit.jupiter.api.Test;
11 import org.springframework.beans.factory.annotation.Autowired;
12 import org.springframework.boot.test.context.SpringBootTest;
13 import org.springframework.boot.test.web.server.LocalServerPort;
14 import org.springframework.test.web.reactive.server.WebTestClient;
15
16 import java.time.LocalDate;
17
18 @SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.
19     RANDOM_PORT)
19 class NextcarApplicationTests {
20
21     @LocalServerPort
22     private int port;
23
24     private WebTestClient webTestClient;
25
26     @Autowired
27     private VeiculoRepository veiculoRepository;
28
29     @Autowired
30     private UsuarioRepository usuarioRepository;
31
32     @Autowired
33     private VendaRepository vendaRepository;
34
```

```
35 @BeforeEach
36 void setup() {
37
38     webTestClient = WebTestClient
39         .bindToServer()
40         .baseUrl("http://localhost:" + port)
41         .build();
42
43     vendaRepository.deleteAll();
44     veiculoRepository.deleteAll();
45     usuarioRepository.deleteAll();
46 }
47
48 @Test
49 void testCreateNextcarUsuarioSuccess() {
50
51     UsuarioRequestDTO usuario = new UsuarioRequestDTO(
52         "Emerson",
53         "emerson@test.com",
54         "123"
55     );
56
57     webTestClient
58         .post()
59         .uri("/usuarios")
60         .bodyValue(usuario)
61         .exchange()
62         .expectStatus().is2xxSuccessful()
63         .expectBody()
64         .jsonPath("$.nome").isEqualTo(usuario.getNome())
65         .jsonPath("$.login").isEqualTo(usuario.getLogin());
66 }
67
68 @Test
69 void testCreateNextcarUsuarioFail() {
70
71     UsuarioRequestDTO usuario = new UsuarioRequestDTO(
72         " ",
73         " ",
74         " "
75     );
76 }
```

```
77     webTestClient
78         .post()
79         .uri("/usuarios")
80         .bodyValue(usuario)
81         .exchange()
82         .expectStatus()
83         .isBadRequest();
84 }
85
86 @Test
87 void testCreateNextcarVeiculoSuccess() {
88
89     Veiculo veiculo = new Veiculo(
90         "Toyota",
91         "Corolla",
92         2020,
93         80000.0,
94         "Sedan",
95         "disponivel",
96         "normal"
97     );
98
99     webTestClient
100         .post()
101         .uri("/veiculos")
102         .bodyValue(veiculo)
103         .exchange()
104         .expectStatus()
105         .is2xxSuccessful()
106         .expectBody()
107         .jsonPath("$.marca").isEqualTo(veiculo.getMarca())
108         .jsonPath("$.modelo").isEqualTo(veiculo.getModelo())
109         .jsonPath("$.ano").isEqualTo(veiculo.getAno())
110         .jsonPath("$.preco").isEqualTo(veiculo.getPreco())
111         .jsonPath("$.tipo").isEqualTo(veiculo.getTipo())
112         .jsonPath("$.status").isEqualTo(veiculo.getStatus())
113         .jsonPath("$.tipoPreco").isEqualTo(veiculo.getTipoPreco());
114 }
115
116 @Test
117 void testCreateNextcarVeiculoFail() {
118
```

```
119     Veiculo veiculo = new Veiculo(  
120         "",  
121         "",  
122         0,  
123         0.0,  
124         "",  
125         "",  
126         ""  
127     );  
128  
129     webTestClient  
130         .post()  
131         .uri("/veiculos")  
132         .bodyValue(veiculo)  
133         .exchange()  
134         .expectStatus()  
135         .isBadRequest();  
136 }  
137  
138 @Test  
139 void testCreateNextcarVendaSuccess() {  
140  
141     Veiculo veiculo = veiculoRepository.save(  
142         new Veiculo(  
143             "Honda",  
144             "Civic",  
145             2022,  
146             120000.0,  
147             "Sedan",  
148             "disponivel",  
149             "normal"  
150         )  
151     );  
152  
153     var usuario = usuarioRepository.save(  
154         new nextcar.model.Usuario(  
155             "Emerson",  
156             "emerson@test.com",  
157             "123"  
158         )  
159     );  
160
```

```
161     VendaRequestDTO venda = new VendaRequestDTO();
162
163     venda.setData(LocalDate.now());
164     venda.setValorFinal(120000.0);
165     venda.setVeiculoId(veiculo.getId());
166     venda.setUsuarioId(usuario.getId());
167
168     webTestClient
169         .post()
170         .uri("/vendas")
171         .bodyValue(venda)
172         .exchange()
173         .expectStatus()
174         .is2xxSuccessful()
175         .expectBody()
176         .jsonPath("$.valorFinal").isEqualTo(120000.0)
177         .jsonPath("$.usuarioNome").isEqualTo("Emerson");
178 }
179
180 @Test
181 void testCreateNextcarVendaFail() {
182
183     VendaRequestDTO venda = new VendaRequestDTO();
184
185     venda.setData(LocalDate.now());
186     venda.setValorFinal(0.0);
187     venda.setVeiculoId(999L);
188     venda.setUsuarioId(999L);
189
190     webTestClient
191         .post()
192         .uri("/vendas")
193         .bodyValue(venda)
194         .exchange()
195         .expectStatus()
196         .is4xxClientError();
197 }
198 }
```

Listing 6: Testes de Integração Avançados — NextcarApplicationTests

5 Log de Refatoração

5.1 Antes vs. Depois

Tabela 26: *Code Smells e Refatorações Realizadas*

	Smell	Localização	Técnica	Resultado
1	Modelo exposto como DTO	UsuarioControlle UsuarioService	Extract Class	A senha deixou de ser exposta nas respostas da API.
2	Exceção genérica	VeiculoService PrecoStrategyFac	Replace Exception	As exceções passaram a representar regras específicas do domínio.
3	@Component em interface	PrecoStrategy	Move Annotation	Os beans passaram a ser definidos apenas nas implementações concretas.
4	Acoplamento excessivo no Service	VendaService	Extract Class (Facade)	O fluxo de venda foi centralizado na VendaFacade; VendaService depende apenas de VendaRepository.
5	Magic String com if/else	VeiculoService	Replace Conditional with Polymorphism	As estratégias passaram a ser resolvidas pela PrecoStrategyFactory usando o padrão Strategy.
6	@Valid ausente no update	VeiculoControlle	Add Validation	A validação de entrada tornou os dados consistentes em todos os endpoints.

5.2 Exemplo 1: Modelo Exposto como DTO

Antes: a entidade `Usuario` era retornada diretamente pelo controller e pelo service, expondo o campo `senha` no JSON de resposta — mesmo criptografado, trata-se de dado sensível que não deve trafegar na API.

```
1 // SMELL: controller retorna a entidade      campo senha aparece no
   JSON
2 @PostMapping
3 public Usuario save(@RequestBody @Valid Usuario usuario) {
4     return service.save(usuario);
5 }
6
7 @GetMapping
8 public List<Usuario> find() {
9     return service.find(); // retorna senha no JSON de cada usuario
10 }
11
12 // Resposta JSON resultante:
13 // { "id": 1, "nome": "Emerson", "login": "...", "senha": "$2a$10$
   ..." }
```

Listing 7: Antes: entidade exposta diretamente na resposta da API

Depois: criados `UsuarioRequestDTO` (entrada) e `UsuarioResponseDTO` (saída). O response DTO não possui campo `senha`, eliminando a exposição do dado sensível e desacoplando o modelo de domínio do contrato da API.

```
1 // UsuarioResponseDTO.java      saída sem campo senha
2 @Getter @Setter
3 public class UsuarioResponseDTO {
4     private Long id;
5     private String nome;
6     private String login;
7     // campo senha AUSENTE
8     public UsuarioResponseDTO(Usuario u) {
9         this.id = u.getId();
10        this.nome = u.getNome();
11        this.login = u.getLogin();
12    }
13 }
14
15 // Controller refatorado
16 @PostMapping
```

```
17 public UsuarioResponseDTO save(@RequestBody @Valid UsuarioRequestDTO
    dto) {
18     return service.save(dto);
19 }
20
21 @GetMapping
22 public List<UsuarioResponseDTO> find() {
23     return service.find();
24 }
25
26 // Resposta JSON resultante:
27 // { "id": 1, "nome": "Emerson", "login": "..."} }
```

Listing 8: Depois: retorno seguro via DTO sem campo senha

5.3 Exemplo 2: Exceção Tipada

Antes: erro genérico lançado com `RuntimeException`, sem tipo específico, sem o valor inválido na mensagem e impossível de capturar isoladamente em testes ou handlers.

```
1 // SMELL: RuntimeException generica      nao tipada, mensagem pobre
2 public Veiculo update(Veiculo veiculoNovo, Long id) {
3     PrecoStrategy strategy = strategies.get(veiculoNovo.getTipoPreco
        ());
4     if (strategy == null) {
5         throw new RuntimeException("Tipo de preco invalido"); // sem
            o tipo!
6     }
7     // ...
8 }
```

Listing 9: Antes: exceção genérica sem informação de domínio

Depois: criada a classe `TipoPrecoInvalidoException` que estende `RuntimeException`, carrega o tipo inválido na mensagem e permite captura específica em testes e em handlers globais de exceção.

```
1 // TipoPrecoInvalidoException.java
2 public class TipoPrecoInvalidoException extends RuntimeException {
3     public TipoPrecoInvalidoException(String tipo) {
4         super("Tipo de preco invalido: " + tipo);
5     }
6 }
7
```



```
8 // PrecoStrategyFactory.java      usa a execucao tipada
9 public PrecoStrategy getStrategy(String tipo) {
10     PrecoStrategy strategy = strategies.get(tipo);
11     if (strategy == null) {
12         throw new TipoPrecoInvalidoException(tipo); // tipo aparece
13             na mensagem
14     }
15     return strategy;
16 }
17
18 // Teste especifico agora possivel
19 @Test
20 void testGetStrategyTipoInvalido() {
21     assertThrows(TipoPrecoInvalidoException.class,
22         () -> factory.getStrategy("invalido"));
23 }
```

Listing 10: Depois: exceção de domínio tipada com mensagem diagnóstica

5.4 Exemplo 3: @Component em Interface

Antes: a interface `PrecoStrategy` possuía a anotação `@Component("normal")` diretamente nela. O Spring não registra beans a partir de interfaces — a anotação era ignorada em runtime, o bean "normal" nunca existia no contexto, e o padrão Strategy falhava silenciosamente.

```
1 // SMELL: @Component em interface      bean nunca e registrado pelo
2     Spring
3 @Component("normal")
4 public interface PrecoStrategy {
5     double calcularPreco(double valor);
6 }
7
8 // PrecoNormal sem anotacao      bean "normal" nao existe no contexto
9 public class PrecoNormal implements PrecoStrategy {
10     @Override
11     public double calcularPreco(double valor) {
12         return valor;
13     }
14 }
```

Listing 11: Antes: @Component incorretamente aplicado na interface

Depois: a anotação foi movida para as implementações concretas. A interface permanece limpa, definindo apenas o contrato de domínio sem responsabilidade de infraestrutura.

```
1 // Interface limpa      sem anotacao de infraestrutura
2 public interface PrecoStrategy {
3     double calcularPreco(double valor);
4 }
5
6 // Bean "normal" registrado corretamente
7 @Component("normal")
8 public class PrecoNormal implements PrecoStrategy {
9     @Override
10    public double calcularPreco(double valor) {
11        return valor;
12    }
13 }
14
15 // Bean "desconto" registrado corretamente
16 @Component("desconto")
17 public class PrecoDesconto implements PrecoStrategy {
18     @Override
19    public double calcularPreco(double valor) {
20        return valor * 0.9;
21    }
22 }
```

Listing 12: Depois: @Component nas implementações concretas

5.5 Exemplo 4: Controller Desacoplado com Facade

Antes: o VendaService dependia de três repositórios (VendaRepository, VeiculoRepository e UsuarioRepository), acumulando responsabilidades de múltiplos domínios e aumentando desnecessariamente seu acoplamento eferente.

```
1 // SMELL: VendaService depende de 3 repositories
2 @Service
3 public class VendaService {
4     private final VendaRepository vendaRepository;
5     private final VeiculoRepository veiculoRepository; // nao e sua
6     // responsabilidade
7     private final UsuarioRepository usuarioRepository; // nao e sua
8     // responsabilidade
9
10    public Venda save(Venda venda) {
11        Veiculo veiculo = veiculoRepository
```

```
10         .findById(venda.getVeiculo().getId())
11         .orElseThrow(() -> new EntityNotFoundException("Veiculo
12             nao encontrado"));
13     Usuario usuario = usuarioRepository
14         .findById(venda.getUsuario().getId())
15         .orElseThrow(() -> new EntityNotFoundException("Usuario
16             nao encontrado"));
17     venda.setVeiculo(veiculo);
18     venda.setUsuario(usuario);
19     return vendaRepository.save(venda);
20 }
21 }
```

Listing 13: Antes: VendaService com acoplamento a três repositórios

Depois: a orquestração foi extraída para o VendaFacade. O VendaService passou a depender apenas de VendaRepository, respeitando o SRP. O controller delega o caso de uso inteiro ao Facade.

```
1 // VendaService      responsabilidade unica: persistencia de Venda
2 @Service
3 public class VendaService {
4     private final VendaRepository vendaRepository; // apenas 1
5         dependencia
6
7     public Venda save(Venda venda) {
8         return vendaRepository.save(venda);
9     }
10 }
11
12 // VendaFacade      orquestra o caso de uso "registrar venda"
13 @Component
14 public class VendaFacade {
15     // dependencias via services especializados, nao repositories
16     // diretos
17
18     public VendaResponseDTO registrarVenda(VendaRequestDTO dto) {
19         Veiculo veiculo = veiculoService.buscarPorId(dto.getVeiculoId
20             ());
21         Usuario usuario = usuarioService.buscarPorId(dto.getUsuarioId
22             ());
23         double preco = strategyFactory
24             .getStrategy(veiculo.getTipoPreco())
25             .calcularPreco(veiculo.getPreco());
26         if (veiculo.getStatus().equalsIgnoreCase("disponivel")) {
```

```
22         veiculo.setStatus("vendido");
23         veiculoService.save(veiculo);
24     }
25     return new VendaResponseDTO(vendaService.save(new Venda(...))
26         );
27 }
28
29 // VendaController        delega ao Facade
30 @PostMapping
31 public VendaResponseDTO criar(@RequestBody @Valid VendaRequestDTO dto
32     ) {
33     return vendaFacade.registrarVenda(dto);
34 }
```

Listing 14: Depois: VendaService simples e VendaFacade orquestrador

5.6 Exemplo 5: Magic String com if/else

Antes: a validação do tipo de preço usava strings literais em um bloco if/else. Para adicionar um novo tipo seria necessário alterar o método, violando o OCP. Além disso, a lógica duplicava implicitamente o conhecimento de quais tipos existem.

```
1 // SMELL: Magic String + if/else        viola OCP, fragil a typos
2 public Veiculo update(Veiculo veiculoNovo, Long id) {
3     Veiculo existe = buscarPorId(id);
4     if (!veiculoNovo.getTipoPreco().equalsIgnoreCase("normal") &&
5         !veiculoNovo.getTipoPreco().equalsIgnoreCase("desconto")) {
6         throw new TipoPrecoInvalidoException(""); // perde o tipo na
7             mensagem
8     }
9     existe.setPreco(veiculoNovo.getPreco()); // nao aplica a strategy
10    // ...
11    return veiculoRepository.save(existe);
12 }
```

Listing 15: Antes: validação com Magic String e if/else

Depois: o bloco if/else foi eliminado. A validação e a seleção do algoritmo foram delegadas à PrecoStrategyFactory. Novos tipos de preço são adicionados criando apenas uma nova implementação de PrecoStrategy, sem alterar o VeiculoService.

```
1 // VeiculoService        sem if/else, sem strings literais
2 @Service
```

```
3 public class VeiculoService {
4     private final VeiculoRepository veiculoRepository;
5     private final PrecoStrategyFactory strategyFactory;
6
7     public Veiculo update(Veiculo veiculoNovo, Long id) {
8         Veiculo existe = buscarPorId(id);
9         // Factory valida e retorna a strategy        lança
10            TipoPrecoInvalidoException
11        double precoFinal = strategyFactory
12            .getStrategy(veiculoNovo.getTipoPreco())
13            .calcularPreco(veiculoNovo.getPreco());
14        existe.setMarca(veiculoNovo.getMarca());
15        existe.setModelo(veiculoNovo.getModelo());
16        existe.setAno(veiculoNovo.getAno());
17        existe.setPreco(precoFinal); // preco calculado pela strategy
18        existe.setTipoPreco(veiculoNovo.getTipoPreco());
19        return veiculoRepository.save(existe);
20    }
21
22    // Para adicionar "premium": apenas nova classe, sem alterar
23    // VeiculoService
24    @Component("premium")
25    public class PrecoPremium implements PrecoStrategy {
26        public double calcularPreco(double valor) { return valor * 1.2; }
27    }
```

Listing 16: Depois: Factory elimina o if/else e aplica o algoritmo correto

5.7 Exemplo 6: @Valid Ausente no update

Antes: o método `update()` do `VeiculoController` não possuía `@Valid`, permitindo que dados inválidos (marca vazia, preço negativo, ano nulo) chegassem ao service sem serem interceptados pelo Bean Validation, criando um comportamento assimétrico em relação ao `save()`.

```
1 // SMELL: save tem @Valid, update nao tem        comportamento
2 // inconsistente
3 @PostMapping
4 public Veiculo save(@RequestBody @Valid Veiculo veiculo) {
5     return service.save(veiculo); // validado corretamente
6 }
```

```
7 @PutMapping("{id}")
8 public Veiculo update(@RequestBody Veiculo veiculo, // SEM @Valid
9                       @PathVariable Long id) {
10     return service.update(veiculo, id); // marca="", preco=-1 passam
11     direto
12 }
```

Listing 17: Antes: @Valid ausente no update — dados inválidos passam

Depois: @Valid adicionado ao método update(), tornando a validação consistente em todos os endpoints. Qualquer violação das constraints da entidade (@NotBlank, @Positive, @NotNull) retorna 400 Bad Request antes de chegar ao service.

```
1 // Validacao consistente em save e update
2 @PostMapping
3 public Veiculo save(@RequestBody @Valid Veiculo veiculo) {
4     return service.save(veiculo);
5 }
6
7 @PutMapping("{id}")
8 public Veiculo update(@RequestBody @Valid Veiculo veiculo, // @Valid
9                       @PathVariable Long id) {
10     return service.update(veiculo, id);
11 }
12
13 // Agora ambos retornam 400 Bad Request para dados invalidos.
14 // Constraints verificadas: @NotBlank (marca, modelo, tipo, status,
15 //                           tipoPreco),
16 //                           @Positive (preco, ano), @NotNull (ano,
17 //                           preco)
```

Listing 18: Depois: @Valid presente em todos os endpoints de escrita

6 Conclusão

O projeto NextCar foi desenvolvido seguindo princípios modernos de Engenharia de Software, com foco em organização, manutenção, escalabilidade e qualidade do código. A aplicação foi estruturada em arquitetura em camadas, separando responsabilidades entre controle, serviço, persistência e modelo de domínio, facilitando a manutenção e evolução do sistema.

Durante o desenvolvimento, foram aplicados DTOs para desacoplamento entre as entidades e as requisições da API, além da utilização de padrões de projeto GoF e princípios SOLID e GRASP para garantir baixo acoplamento, alta coesão e melhor reutilização de código. O

sistema também passou por etapas de refatoração para melhoria estrutural e redução de redundâncias, mantendo compatibilidade com os requisitos funcionais definidos inicialmente.

A modelagem UML foi utilizada para representar os principais componentes e relacionamentos do sistema, auxiliando no planejamento da arquitetura e na validação das regras de negócio. Além disso, foram implementados testes automatizados para validação das funcionalidades principais, contribuindo para maior confiabilidade e estabilidade da aplicação.

Síntese Final

O projeto demonstra domínio de análise e projeto de software ao integrar levantamento de requisitos, modelagem UML, arquitetura em camadas, persistência de dados, aplicação de padrões de projeto, princípios SOLID/GRASP, testes automatizados e refatorações estruturais. O resultado é uma aplicação organizada, modular e preparada para manutenção e expansão futura.

Repositório Oficial

Com o objetivo de garantir transparência, rastreabilidade e versionamento contínuo do desenvolvimento, todo o código-fonte do projeto NextCar encontra-se disponível em seu repositório oficial no GitHub. O repositório contempla a implementação da aplicação, documentação técnica, Arquitetura C4(Model), fluxo do Sistema e demais artefatos produzidos durante o desenvolvimento do sistema.

<https://github.com/NextCar-Project/projeto-next-car>